

Estrategia Avanzada de Construcción: Imágenes Multifase y Reducción de la Superficie de Ataque

En un contexto de ciberseguridad, la eficiencia del *build* y la minimización del tamaño de la imagen son fundamentales para reducir la superficie de ataque y asegurar un despliegue rápido. La técnica de las **Imágenes Multifase** (*Multistage Builds*) es la solución recomendada para lograr estos objetivos.

I. Concepto y Beneficios de las Imágenes Multifase

Una imagen multifase de Docker permite definir **múltiples etapas o fases de construcción** dentro de un único Dockerfile, separando el **entorno de compilación** (grande y con herramientas) del **entorno de ejecución** (mínimo y limpio).

¿Qué es una Imagen Multifase?

En lugar de tener un Dockerfile que intenta instalar dependencias de construcción y de *runtime* en la misma imagen, se utiliza un enfoque de "construir aquí, desplegar allí":

1. **Fase de Construcción (Build):** Utiliza una imagen grande que contiene todas las herramientas necesarias (compiladores, SDKs, *package managers*).
2. **Fase de Ejecución (Runtime):** Utiliza una imagen base mínima y solo copia los **artefactos esenciales** (el binario compilado o los archivos de la aplicación) desde la fase de construcción.

Beneficios Clave para la Seguridad y el Rendimiento

- **Seguridad:** Al excluir herramientas de construcción innecesarias (como compiladores, *shell* de la imagen, cabeceras de desarrollo), se **minimiza la superficie de ataque**. La eliminación de la *shell* de la imagen, en particular,

reduce el riesgo de explotación de vulnerabilidades de ejecución remota de código (RCE) que requieran acceso a `/bin/bash` o `/bin/sh`.

- **Tamaño Reducido (Lightweight):** Se crean imágenes mucho más livianas, ya que solo incluyen las dependencias y artefactos finales. Esto conduce a descargas, despliegues y almacenamiento más rápidos en los registros.
 - **Construcciones Más Rápidas:** Se optimiza el proceso de *build* al reconstruir solo las etapas necesarias cuando se producen cambios. Docker puede reutilizar la caché de etapas previas que no han cambiado, mejorando el tiempo total de compilación, especialmente en proyectos complejos.
 - **Mantenimiento Mejorado:** El Dockerfile se simplifica y se gestiona mejor, ya que consolida el proceso de *build* y *runtime* en un solo archivo de una manera transparente y ordenada.
-

II. Demostración Práctica: Compilando una Aplicación Go

Usaremos una aplicación simple escrita en **Go** (un lenguaje compilado) para demostrar la diferencia de tamaño, ya que el binario de Go, al ser estáticamente compilado, incluye todas sus dependencias y es altamente portable.

Paso 1: Dockerfile Tradicional (Una Sola Etapa)

Inicialmente, la imagen se construye en una sola etapa, que incluye el compilador y todo el SDK.

Dockerfile (Etapa Única - No Optimizado):

```
FROM golang:1.22
```

```
WORKDIR /go/src/app
```

```
COPY . .
```

```
RUN CGO_ENABLED=0 go build -o /app .
```

```
CMD ["/app"]
```

- Se utiliza la imagen **golang:1.22** como base, la cual es grande porque contiene todas las utilidades de Go.
- Se compila la aplicación (**main.go**) y se genera el binario ejecutable (**/app**).
- La imagen final contiene el binario **más** el compilador y todo el SDK de Go.

Resultado de Tamaño (ejemplo): 896 MB (como se ve en la salida de **docker image ls**).

Paso 2: Implementación de la Imagen Multifase

Para optimizar, definimos dos fases separadas: la fase de **build** (compilación) y la fase final de **runtime** (ejecución).

Dockerfile (Multifase - Optimizado):

```
# -- FASE 1: CONSTRUCCIÓN (BUILD) --
```

```
FROM golang:1.22 AS build
```

```
WORKDIR /go/src/app
```

```
COPY . .
```

```
RUN CGO_ENABLED=0 go build -o /app . # Genera el binario  
estático /app
```

```
# -- FASE 2: EJECUCIÓN (RUNTIME) --
```

```
FROM ubuntu # Base pequeña, solo para ejecutar el binario
```

```
COPY --from=build /app /app # ¡Copia solo el binario desde la  
fase 'build'!
```

```
CMD ["/app"]
```

Explicación de las Fases:

1. **Fase 1 (AS build):** Esta fase tiene el compilador (`golang:1.22`). El binario `/app` se compila aquí. El uso de `AS build` nombra la fase para que pueda ser referenciada posteriormente. Todo el contenido de esta fase (compilador, código fuente, etc.) se **ignora** en la imagen final.
2. **Fase 2 (FROM ubuntu):** Esta es la imagen final que se genera. Utiliza una imagen base mínima (ej. `ubuntu`).
3. **Copia Dirigida:** El comando clave es `COPY --from=build /app /app`. El *flag* `--from=build` le dice a Docker que el archivo de origen (`/app`) se encuentra en el sistema de archivos de la fase nombrada `build`. Solo el binario esencial se transfiere a la imagen final.

Construcción y Verificación (Etiqueta `multi`):

```
docker build -t mygoapp:multi .
```

```
docker run -it -p 8080:8080 mygoapp:multi
```

```
curl localhost:8080 # La aplicación sigue funcionando.
```

Resultado de Tamaño (ejemplo): 84.9 MB (en comparación con los 896 MB originales, una reducción significativa).

Paso 3: Aumento de la Seguridad con Imágenes Distroless

Para maximizar la seguridad, se puede ir un paso más allá utilizando imágenes **Distroless** (un proyecto de Google GCR). Las imágenes Distroless contienen **solo lo mínimo necesario** para ejecutar la aplicación y sus dependencias de *runtime*, y, crucialmente, **no incluyen una shell** (`/bin/bash` o `/bin/sh`).

Beneficio de Seguridad: Si un atacante logra explotar una vulnerabilidad RCE, a menudo el ataque fallará al intentar ejecutar una *shell* para establecer una conexión de control.

Dockerfile (Distroless - Máxima Seguridad):

```
# -- FASE 1: CONSTRUCCIÓN (BUILD) --

FROM golang:1.22 AS build

WORKDIR /go/src/app

COPY . .

RUN CGO_ENABLED=0 go build -o /app .


# -- FASE 2: EJECUCIÓN (RUNTIME) --

FROM gcr.io/distroless/static # Imagen mínima, sin shell

COPY --from=build /app /app

CMD ["/app"]
```

Construcción y Verificación (Etiqueta **distroless**):

```
docker build -t mygoapp:distroless .
```

```
docker run -it -p 8080:8080 mygoapp:distroless
```

```
curl localhost:8080 # La aplicación sigue funcionando.
```

Resultados de la Comparativa de Imágenes

El comando `docker image ls` ilustra claramente los beneficios de las imágenes multifase y Distroless:

REPOSITO RY	TAG	SIZE (Ejemplo)	BASE	BENEFICIO
mygoapp	latest (Etapa Única)	896 MB	golang:1.22	Alto consumo de recursos y mayor superficie de ataque.
mygoapp	multi (Multifase)	84.9 MB	ubuntu	Gran reducción de tamaño; aún incluye una <i>shell</i> de Ubuntu.
mygoapp	distrol ess (Multifase)	9.6 MB	gcr.io/distroles s/static	Reducción de tamaño máxima (casi 90 veces más pequeña); sin shell , minimizando el riesgo de RCE.

El uso de **imágenes multifase** se considera una **buena práctica altamente recomendable** en el desarrollo de contenedores. Permite incorporar únicamente los archivos esenciales, reduciendo drásticamente el tamaño y mejorando inherentemente la seguridad de la imagen final.

Mejora de la Información y Flexibilidad: LABEL y ARG en el Dockerfile

Continuando con las técnicas avanzadas para la construcción de imágenes seguras y mantenibles, este capítulo detalla el uso de **metadatos** (**LABEL**) para documentar las imágenes y las **variables de construcción** (**ARG**) para añadir flexibilidad a los Dockerfiles.

I. LABEL: Añadiendo Metadatos a la Imagen

El comando **LABEL** permite incrustar **metadatos** o información descriptiva dentro de la imagen final. Estos metadatos son pares de clave-valor que proporcionan contexto sobre la imagen, lo que se considera una buena práctica para el mantenimiento y la documentación.

Sintaxis y Propósito

- **Sintaxis:**
`LABEL <clave>=<valor>`
- **Propósito:** Proporciona información descriptiva para los usuarios de la imagen, como su **versión**, el **mantenedor**, la **descripción**, o el **proveedor**. Si bien las etiquetas pueden ser arbitrarias, se recomienda seguir convenciones comunes.
- **Visibilidad:** Las etiquetas son visibles cuando se **inspecciona** la imagen usando el comando `docker inspect`.

Reglas de Aplicación y Buenas Prácticas

1. **Persistencia en la Fase Final:** En un **Dockerfile multifase**, los **LABEL** solo se aplican a la imagen final si se definen en la **última fase** de construcción.

- **Ejemplo Práctico:** Si se define `LABEL stage="build"` en la primera fase (`AS build`), esta etiqueta **no** aparecerá en la imagen final, ya que el motor de Docker descarta la información de las fases intermedias. La etiqueta debe colocarse en la fase final:

```
FROM gcr.io/distroless/static
```

```
LABEL stage="final" # Esta etiqueta se aplica a la imagen final.
```

2. **Combinación de Etiquetas y Reducción de Capas:** Para mejorar el rendimiento y evitar la creación de múltiples capas innecesarias (aunque las etiquetas no causan problemas graves de rendimiento), se recomienda **combinar múltiples etiquetas** en una sola instrucción `LABEL`.

- **Sintaxis Recomendada (Múltiples Etiquetas):**

```
LABEL stage="final" \

    maintainer="Encargado de mantener la imagen"
```

- **Verificación de Metadatos:** Tras reconstruir la imagen, se comprueba la inclusión de las etiquetas utilizando la CLI:

```
docker inspect mygoapp
```

```
# En la sección "Config.Labels" se verán:
```

```
# "Labels": { "stage": "final", "maintainer": "..." }
```

3. **Seguridad y Contacto:** Añadir metadatos como el `maintainer` es una buena práctica de mantenimiento, ya que proporciona un punto de contacto

fácil para cualquier persona que necesite reportar un problema o una vulnerabilidad en la imagen.

II. ARG: Variables de Construcción

El comando **ARG** permite definir **variables** que pueden ser utilizadas en el Dockerfile **únicamente durante el proceso de construcción**. Estas variables no se persisten en la imagen final, a diferencia de las variables de entorno (**ENV**, que veremos más adelante).

Sintaxis y Propósito

- **Sintaxis:**
`ARG <nombre_variable>[=<valor_por_defecto>]`
- **Propósito:** Se utiliza para añadir **flexibilidad** al Dockerfile, permitiendo modificar parámetros clave (como la versión de una imagen base, un *token*, o una *flag* de compilación) sin tener que editar el archivo directamente.
- **Sobreescritura:** El valor de la variable **ARG** puede ser sobrescrito en tiempo de construcción utilizando el parámetro **--build-arg** del comando **docker build**:
`docker build -t mi-app --build-arg VERSION=1.24 .`

Reglas de Visibilidad y Ámbito

La visibilidad de las variables **ARG** está estrictamente limitada por la estructura del Dockerfile, lo que es un aspecto crucial a entender:

1. **Visibilidad Global Inicial (FROM):** La instrucción **ARG** es la **única** que puede aparecer **antes de la instrucción FROM**. Si se define aquí, su valor está disponible para ser usado en el **FROM**.

- **Uso Típico:** Definir la versión de la imagen base:

```
ARG VERSION=1.22
```

```
FROM golang:$VERSION as build # $VERSION se resuelve  
aquí.
```

2. **Visibilidad Limitada a la Fase (RUN, COPY, etc.):** Una vez que el Dockerfile pasa el **FROM**, las variables definidas antes del **FROM** **ya no son visibles** en las instrucciones subsiguientes (**RUN**, **COPY**, **CMD**, etc.) a menos que se **redefinan** dentro de la fase (bloque **FROM...**).

- **Ejemplo Práctico de Ambigüedad:** Si se intenta imprimir el valor de **\$VERSION** después del **FROM** sin redefinirlo:

```
# ARG VERSION=1.22 (Global)
```

```
FROM golang:$VERSION as build
```

```
RUN echo $VERSION # <-- ¡Esto imprime el texto literal  
"$VERSION", no el valor 1.22!
```

3. **Redefinición en Bloque:** Para usar una variable **ARG** en instrucciones dentro de un bloque de fase (o en fases subsiguientes), **debe definirse nuevamente** dentro de ese bloque:

- **Ejemplo para Visibilidad Interna:**

```
ARG GLOBAL_VERSION=1.22
```

```
FROM golang:$GLOBAL_VERSION as build
```

```
ARG VERSION=1.20 # Re-definida para esta fase
```

```
RUN echo $VERSION # <-- ¡Esto imprime el valor "1.20"!
```

- Esta regla es importante porque si una variable **ARG** afecta al **FROM**, cualquier valor que se le asigne en un bloque **FROM** posterior sobrescribirá el valor de esa variable solo para ese bloque.

Conclusión sobre **ARG**

El uso de **ARG** es altamente recomendado para cualquier valor específico que pueda necesitar cambiar sin modificar el código fuente del Dockerfile, pero el desarrollador debe ser consciente de sus **estrictas reglas de visibilidad** para garantizar que los valores se utilicen correctamente en el **contexto de construcción** deseado.

OLD

Distribución y Buenas Prácticas de Imágenes

Docker: Registros y Nomenclatura

Una vez que se han optimizado y asegurado las imágenes de Docker, el siguiente paso es la **distribución**. Las imágenes creadas localmente no son útiles a escala.

El **Registro de Docker (Registry)** es el componente esencial que permite a los equipos compartir, distribuir y gestionar imágenes a través de diferentes entornos de desarrollo, prueba y producción.

I. El Registro de Docker (Registry)

Un Registro de Docker es una aplicación del lado del servidor que actúa como un sistema de almacenamiento centralizado para imágenes.

Características Clave

1. **Almacenamiento (Storage):** Almacena las imágenes de Docker junto con sus metadatos asociados, como etiquetas (**tags**) y *hashes* de capa.
2. **Distribución (Distribution):** Proporciona la infraestructura para distribuir imágenes de forma correcta y eficiente entre diversos entornos (Desarrollo, QA, Producción).
3. **Autenticación y Autorización:** Implementa mecanismos para controlar quién puede subir (**push**) y descargar (**pull**) imágenes, especialmente para repositorios privados.
4. **Búsqueda y Descubrimiento:** Permite a los usuarios buscar y descubrir imágenes almacenadas.
5. **Replicación:** Algunos registros implementan la replicación en múltiples instancias para garantizar la redundancia y mejorar la disponibilidad global.

Registros Populares

- **Docker Hub ([docker.io](https://hub.docker.com/)):** El registro oficial y por defecto de Docker. Es la cuenta [library](https://hub.docker.com/u/library/) donde se encuentran las imágenes oficiales (Ubuntu, Nginx, Python, etc.).
- **ECR:** Elastic Container Registry (en AWS).
- **GCR:** Google Container Registry (Google tiene su propio registro).
- **ACR:** Azure Container Registry (en Microsoft Azure).

II. Nomenclatura y Etiquetas ([Tags](#)) de Imágenes

Cuando construimos una imagen localmente, la nombramos con un nombre simple (ej. [mygoapp](#)). Sin embargo, para la distribución, la imagen debe seguir un formato específico que incluya la referencia al registro.

Formato Completo de Nomenclatura

El formato completo de una imagen distribuible incluye la ubicación y la versión específica:

[\[registro-dominio:puerto\]/\[usuario/cuenta\]/\[nombre-imagen\]:\[etiqueta\]](#)

- **Registro por Defecto:** Si el [\[registro-dominio:puerto\]](#) se omite, Docker asume por defecto [docker.io](#).
- **Uso de Imágenes Oficiales:** Cuando se hace `docker pull golang:1.22`, Docker resuelve la solicitud a [docker.io/library/golang:1.22](#), donde [library](#) es la cuenta que aloja las imágenes oficiales.
- **Imágenes Privadas/Externas:** Si la imagen no está en Docker Hub, se debe especificar el registro, como se hizo con las imágenes *distroless*: [gcr.io/distroless/static](#).

La Importancia de las Etiquetas (Tags)

La **etiqueta (tag)** es la parte de la nomenclatura que identifica de manera **única** una versión específica, contenido o propósito de una imagen. Se añade al nombre de la imagen y se separa por dos puntos (:).

Buenas Prácticas de Etiquetado

1. **Ser Descriptivo:** Utilice etiquetas que proporcionen información clara sobre la versión y el contenido (ej. **v2.1-alpine**, **20250501**).
2. **Utilizar Versiones Semánticas:** Divida las versiones en partes que identifiquen cambios mayores, menores y parches (ej. **v1.0.1**).
3. **Evitar Etiquetas Genéricas (Especialmente **latest**):** La etiqueta **latest** puede resultar confusa, ya que no necesariamente significa "la imagen más recientemente creada", sino simplemente la última imagen que fue etiquetada como tal. **No garantiza la última versión del código** y puede llevar a inconsistencias si no se gestiona correctamente.
4. **No Reutilizar Etiquetas (Inmutabilidad): Es una muy mala práctica sobrescribir una etiqueta existente** (ej. reconstruir una nueva imagen y volver a etiquetarla como **v2**). Si alguien está utilizando la **v2** en producción y se sobrescribe, su despliegue puede romperse o comportarse de manera inesperada.
5. **Imágenes Inmutables:** Para garantizar que el despliegue siempre use la misma imagen, lo más seguro es referenciarla directamente por su **hash o ID de imagen** (SHA-256), en lugar de por su etiqueta.

III. Gestión de Credenciales y Distribución

Para interactuar con repositorios privados, se requiere autenticación, un proceso que debe ser manejado de forma segura.

1. Autenticación (**docker login**)

El comando **docker login** se utiliza para autenticarse en un registro de contenedores (por defecto, Docker Hub) para poder descargar o subir imágenes privadas.

- **Sintaxis General:**

```
docker login [SERVER] -u [username]
```

- Si *[SERVER]* se omite, se asume *docker.io*.

- **Método Seguro de Contraseña:** Para evitar que la contraseña o el token de acceso personal queden registrados en el historial de comandos

(*~/.bash_history*) o sean visibles por otros procesos del sistema (*ps aux*), se recomienda usar **--password-stdin**:

```
echo "mi_token_secreto" | docker login --username mi_usuario  
--password-stdin
```

- **Token de Acceso Personal:** Es altamente recomendable usar un **Token de Acceso Personal** generado en la configuración de seguridad del registro (ej. Docker Hub) en lugar de la contraseña de la cuenta para las interacciones con la CLI. El token permite revocar permisos fácilmente y limita la exposición de la contraseña principal.

2. Subida de Imágenes (**docker push**)

Para que una imagen local (creada con **docker build**) esté disponible en el registro, debe ser **etiquetada correctamente** y luego subida.

- **Problema de Subida:** Si se intenta subir una imagen sin prefijo de cuenta o registro (ej. **docker push mygoapp**), Docker intentará subirla al repositorio **docker.io/library/mygoapp**, lo cual resulta en un error de **acceso denegado** porque **library** está reservada para imágenes oficiales.

- **Solución 1: Construir con Nomenclatura Completa:**

```
docker build -t cyberhades/mygoapp .
```

- **Solución 2: Re-etiquetar (Tagging):** Se utiliza el comando `docker tag` para añadir una etiqueta que incluya la cuenta de destino a una imagen ya existente.

```
docker tag mygoapp:v2 cyberhades/mygoapp:latest
```

- *Este comando crea un nuevo puntero (etiqueta) que apunta al mismo ID de imagen (hash), sin duplicar el binario.*

- **Subida al Registro:**

```
docker push cyberhades/mygoapp:v2
```

```
docker push cyberhades/mygoapp:latest
```

- *Se deben subir todos los tags que se quieran persistir en el registro.*

IV. Recapitulación de Buenas Prácticas para la Construcción de Imágenes Seguras

Este módulo concluye con un resumen de las prácticas esenciales que garantizan la seguridad, eficiencia y mantenibilidad de los entornos contenerizados:

1. **Uso de Registros (Distribución):** Utilizar registros de imágenes es la **forma ideal y recomendada** para la distribución de imágenes.
2. **Minimización del Tamaño y de la Superficie de Ataque:**
 - **Imágenes Multifase:** Altamente recomendado. Permite incluir únicamente el binario de ejecución, excluyendo compiladores y herramientas de desarrollo.
 - **Imágenes Base Mínimas (Distroless/Alpine):** Reducir el tamaño de la imagen final (ahorro de almacenamiento y ancho de banda) y eliminar utilidades de *shell* y librerías innecesarias, lo que **mejora la seguridad** al reducir las herramientas disponibles para un atacante en caso de compromiso.

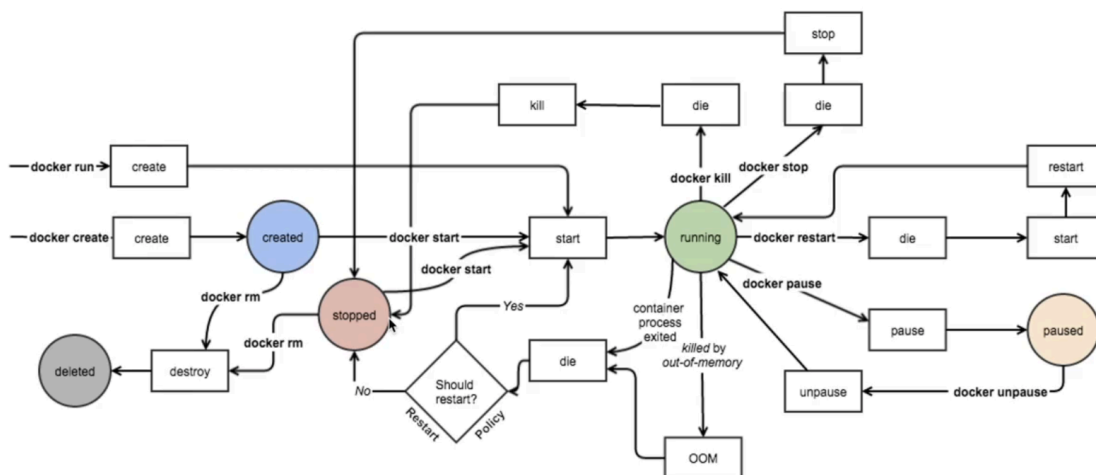
3. Optimización del *Build*:

- **Uso Estratégico de la Caché:** El orden de las instrucciones en el Dockerfile debe ser estratégico (cosas que cambian menos arriba, cosas que cambian más abajo) para aprovechar la caché.
- **Agrupación de Comandos RUN:** Agrupar comandos relacionados con `&&` para reducir el número de capas y mejorar el rendimiento del *runtime* (menos sobrecarga de OverlayFS).
- **.dockerignore:** Utilizar el fichero `.dockerignore` para reducir el contexto de construcción y mejorar el rendimiento.

4. Mantenibilidad y Documentación:

- **Imágenes Base Oficiales/Mantenidas:** Usar imágenes base oficiales o mantenidas internamente por equipos de la empresa.
- **Etiquetado Correcto:** Asignar etiquetas claras y descriptivas a cada versión y ser consciente de la función de la etiqueta `latest`.
- **Control de Versiones:** Los Dockerfiles deben formar parte del repositorio de código centralizado.
- **Metadatos:** Usar la instrucción `LABEL` para añadir información de contacto y descripción de la imagen.

Diagrama de Flujo de Vida y Estados de los Contenedores Docker



El esquema que se proporciona es un **Diagrama de Flujo de Estados (State Flow Diagram)** que ilustra cómo un contenedor Docker se mueve entre diferentes estados a lo largo de su ciclo de vida, basado en los comandos de Docker o eventos internos del sistema.

Aquí tienes una explicación detallada de cada estado y las transiciones que lo conectan:

1. Estado **created** (Creado)

- **¿Cómo se llega?**
 - Mediante los comandos **docker run** o **docker create**. Ambos inician el proceso de creación del contenedor.
- **Características:** El contenedor ha sido construido con todos sus metadatos y configuración, pero el proceso principal aún no se ha iniciado.
- **Transiciones de salida:**
 - Hacia **start (Ejecución)**: Usando el comando **docker start**.

- Hacia **stopped (Detenido)**: Esto ocurre si hay un error inmediato al intentar iniciar el contenedor o si se destruye antes de iniciar.
 - Hacia **deleted (Eliminado)**: Usando **docker rm** (que ejecuta la acción interna **destroy**).
-

2. Estado **start / running** (En Ejecución)

- **¿Cómo se llega?**
 - Desde **created** o **stopped** con el comando **docker start**.
 - Directamente desde **docker run**.
 - **Características**: Es el estado activo; el proceso principal del contenedor se está ejecutando.
 - **Transiciones de salida (Comandos manuales)**:
 - Hacia **die (Muerte)**: Usando **docker kill** (que envía una señal **SIGKILL** o similar). El estado **die** conduce a **stopped**.
 - Hacia **stop (Parada)**: Usando **docker stop** (que primero intenta enviar una señal **SIGTERM** y luego **SIGKILL**). El estado **stop** conduce a **die**, y luego a **stopped**.
 - Hacia **paused (Pausado)**: Usando **docker pause** (que congela los procesos, pero el contenedor sigue "vivo").
 - **Transiciones de salida (Eventos internos)**:
 - Hacia **die**: Si el **proceso del contenedor sale** por sí mismo (terminación normal o error).
 - Hacia **OOM (Out-Of-Memory)**: Si el contenedor es **terminado por el kernel** debido a falta de memoria, lo que también conduce a **die**.
 - Hacia **die / start**: Usando **docker restart** (lo mata y luego lo inicia de nuevo).
-

3. Estado **paused** (Pausado)

- ¿Cómo se llega?
 - Desde **running** con el comando **docker pause**.
 - **Características:** El contenedor existe y su estado se mantiene, pero todos sus procesos están temporalmente detenidos.
 - **Transiciones de salida:**
 - Hacia **running**: Usando **docker unpause**.
-

4. Estado **stopped** (Detenido)

- ¿Cómo se llega?
 - Desde **running** (después de pasar por **die** o **OOM**).
 - Desde **created** (en caso de errores de inicio).
 - **Características:** El proceso principal ha terminado (o ha sido detenido), pero el contenedor y su sistema de archivos aún existen. Es el estado en "reposo".
 - **Transiciones de salida:**
 - Hacia **start** (Ejecución): Usando **docker start**.
 - Hacia **die** / **start**: Si la **Política de Reinicio (Restart Policy)** del contenedor es "Sí", se reinicia automáticamente (vuelve a **start**).
 - Hacia **deleted** (Eliminado): Usando **docker rm** (que ejecuta **destroy**).
-

5. Estado **deleted** (Eliminado)

- ¿Cómo se llega?
 - Desde **stopped** o **created** con el comando **docker rm**.
- **Características:** El contenedor y todos sus recursos han sido eliminados del sistema Docker. Es el final del ciclo de vida.

(Nota: La acción intermedia **die** es un evento de transición que ocurre justo antes de que el contenedor pase al estado final **stopped** y registra el código de salida.)

Seguridad y Acceso a Contenedores: Gestión de Puertos y Redes

En el entorno de un Máster en Ciberseguridad, comprender cómo Docker gestiona el **aislamiento de red** y la **exposición de puertos** es esencial. Los contenedores, por defecto, se ejecutan en una red interna privada, aislados del *host* y del mundo exterior. Acceder a las aplicaciones que se ejecutan dentro de ellos requiere técnicas específicas de mapeo o pertenencia a la misma red.

I. Aislamiento de Red y Acceso Interno

Cuando se ejecuta un contenedor sin especificar configuraciones de red, Docker lo asigna a una red virtual interna llamada **bridge**.

1. Red de Aislamiento por Defecto

Comando de Ejecución (Inicial):

```
docker run -it mygoapp
```

- *El contenedor se ejecuta, pero la aplicación no es accesible directamente desde el host.*
- **Inspección del Contenedor:** Para determinar la dirección IP asignada, se puede inspeccionar el contenedor.
 1. **Obtener Nombre/ID:** Utilice `docker container ls` o `docker ps` para ver el nombre asignado (ej. `vigilant_chatterjee`).

Inspeccionar la Red:

```
docker inspect vigilan_t_chatterjee
```

2. *El resultado de la inspección, bajo la sección **Networks**, muestra que el contenedor está conectado a la red **bridge** y tiene asignada una IP interna (ej. `172.17.0.2`).*

Acceso Directo (Solo Linux Nativo): En entornos de Linux con Docker nativo (sin máquina virtual intermedia), es posible acceder a la aplicación utilizando la IP interna del contenedor:

```
curl 172.17.0.2:8080
```

- *En este escenario, el host y el contenedor comparten un contexto de red que permite la comunicación directa a través de la IP interna de la red **bridge**.*

2. Limitaciones de Docker Desktop (Windows/macOS)

Si se utiliza **Docker Desktop** (que opera a través de una máquina virtual), el acceso directo a la IP interna del contenedor (ej. **172.17.0.2**) desde el *host* **falla** debido a la capa de aislamiento de la máquina virtual.

- **Solución 1: Comunicación Contenedor a Contenedor:** La forma de acceder al servicio es arrancando **otro contenedor dentro de la misma red **bridge****.

1. Arrancar Contenedor de Acceso (Ej. Ubuntu):

```
docker run -it ubuntu
```

2. Instalar Utilidad y Acceder (Desde el Contenedor Ubuntu):

```
apt update && apt install curl -y
```

```
curl 172.17.0.2:8080 # El acceso es exitoso entre  
contenedores en la misma red.
```

- *Aunque funcional, esta solución es laboriosa y no es práctica para el desarrollo o la prueba.*
-

II. Mapeo de Puertos (*Port Mapping*): Acceso Externo

La forma estándar y recomendada de acceder a un servicio en un contenedor es **mapear el puerto** del contenedor a un puerto disponible en la interfaz de red del *host* (localhost).

1. Mapeo Explícito y Personalizado (-p)

El *flag* **-p** se utiliza para definir explícitamente qué puerto del *host* se conecta a qué puerto del contenedor.

- **Sintaxis:** **-p <Puerto_Host>:<Puerto_Contenedor>**
- **Propósito:** Permite acceder a la aplicación a través de la interfaz del *host*, utilizando un puerto que puede ser distinto al puerto interno del contenedor.
- **Ejemplo Detallado:** Mapear el puerto **8080** del contenedor al puerto **8000** del *host*:

```
docker run -it -p 8000:8080 mygoapp
curl localhost:8000 # Acceso a través del puerto
mapeado.
```

- **Mapeo de Rangos de Puertos:** Es posible mapear rangos completos de puertos con una sola instrucción:

```
docker run -it -p 8000-8005:8080-8085 mygoapp
```

 - *Docker mapea secuencialmente: Host 8000 a Contenedor 8080, Host 8001 a Contenedor 8081, y así sucesivamente.*

2. Mapeo Automático de Puertos (EXPOSE y -P)

Existe una forma automatizada de exponer los puertos definidos por el desarrollador.

- **EXPOSE (Instrucción de Dockerfile):** Esta instrucción funciona como **metadato** o documentación. Le indica a Docker qué puertos tiene intención de usar el contenedor.
 - **Sintaxis:** **EXPOSE 8080**

- **Efecto:** El comando **EXPOSE** no publica automáticamente el puerto en el *host*. Su función principal es informativa (para otros desarrolladores) y necesaria para el mapeo automático.
 - **-P (Publicar Todos los Puertos Expuestos):** El *flag* **-P** (mayúscula) en **docker run** ordena a Docker que busque todas las instrucciones **EXPOSE** en el Dockerfile y mapee cada uno de esos puertos a un **puerto aleatorio y disponible** en el *host*.
 - **Ejecución:**
`docker run -it -P mygoapp`
 - **Verificación:** Al ejecutar **docker ps**, la salida mostrará el mapeo dinámico, por ejemplo:
0.0.0.0:55000->8080/tcp
 - *Esto significa que el puerto 8080 del contenedor ha sido mapeado al puerto aleatorio 55000 del host.*
-

III. Uso de la Red del *Host* (**--network host**)

Docker ofrece la opción de conectar directamente un contenedor a la pila de red del *host*.

- **Sintaxis:**
`docker run -it --network host mygoapp`
- **Efecto:** El contenedor ya no utiliza la red **bridge** virtual; en su lugar, utiliza la red del sistema operativo del *host*. Los puertos del contenedor se vuelven accesibles directamente a través de la interfaz de red del *host*.
 - **Acceso:** Si la aplicación escucha en el puerto **8080** dentro del contenedor, será accesible en **localhost:8080** en el *host* (solo funciona con Docker en Linux nativo, no en Docker Desktop).

Riesgos de Seguridad y Conclusiones

El uso de `--network host` se considera una mala práctica por varias razones de seguridad y operatividad:

1. **Aislamiento Roto:** El contenedor pierde el aislamiento de red que proporciona Docker. Puede acceder a servicios que solo deberían ser accesibles desde el *host* o viceversa.
2. **Riesgo de Colisión de Puertos:** Si se intenta ejecutar un segundo contenedor que también usa `--network host` y escucha en el puerto `8080`, Docker generará un error de "puerto ya ocupado" (`port already in use`).
3. **Seguridad:** Al compartir el *namespace* de red, cualquier vulnerabilidad en el contenedor podría comprometer directamente la red del sistema *host*.

Conclusión Final:

Los contenedores se ejecutan por defecto en redes aisladas internas (`bridge`). Para garantizar la seguridad y la funcionalidad, el método recomendado para el acceso es siempre el **mapeo de puertos (-p)**, que permite un control estricto sobre qué servicios se exponen al *host* y en qué puertos. Evitar el uso de `--network host` es crucial para mantener la integridad del aislamiento de seguridad proporcionado por Docker.

Gestión de Entornos Múltiples: Los Contextos de Docker

En ciberseguridad y entornos de desarrollo complejos, es habitual interactuar con múltiples entornos de contenedores: la instalación local, un servidor de desarrollo remoto, un clúster de prueba o un entorno de producción. Los **Contextos de Docker** son la característica que permite al cliente Docker (CLI) conectarse y administrar diferentes *Docker Daemons* (servicios) de forma transparente.

I. Introducción a los Contextos de Entornos de Docker

Un **Contexto de Docker** es un conjunto de configuraciones que definen la forma en que el cliente (`docker CLI`) debe conectarse a un *Docker Daemon* específico.

Propósito y Aplicaciones

- **Interacción con Múltiples Entornos:** Los contextos permiten al usuario cambiar fácilmente entre diferentes tipos de entornos Docker:
 - Instalación local nativa (el *daemon* por defecto).
 - Host de Docker remoto (servidor en la nube o máquina virtual).
 - Clústeres de orquestación (Docker Swarm o Kubernetes).
 - El *daemon* virtual de Docker Desktop.
- **Transparencia:** Una vez que se selecciona un contexto, todos los comandos de Docker (`docker build`, `docker run`, `docker image ls`) se dirigen automáticamente al *daemon* de ese contexto, sin que el usuario tenga que especificar la dirección remota o las credenciales en cada comando.

Contextos Vistos Anteriormente (Docker Desktop)

Al instalar **Docker Desktop**, se ejemplifica el concepto de contexto:

1. **Contexto Nativo (default):** El *daemon* que se ejecuta directamente en el sistema operativo host (si es Linux).
2. **Contexto Docker Desktop:** Un contexto creado automáticamente que apunta al *daemon* que se ejecuta dentro de la máquina virtual que instala Docker Desktop.
 - Al arrancar Docker Desktop, el cliente cambia automáticamente al contexto de la máquina virtual.
 - Al cerrar Docker Desktop, revierte el contexto al nativo.

Comandos de Gestión de Contextos

La interacción con los contextos se realiza a través del subcomando **docker context**:

Comando	Propósito
<code>docker context create</code>	Crea una nueva configuración de contexto.
<code>docker context list / ls</code>	Lista los contextos disponibles.
<code>docker context show</code>	Muestra el contexto actualmente activo.
<code>docker context use</code>	Establece un contexto como el activo (con el que trabajará la CLI).

<code>docker context inspect</code>	Muestra información detallada sobre un contexto.
<code>docker context rm</code>	Elimina uno o más contextos.

II. Procedimiento Práctico: Conexión a un Host Remoto

El ejemplo más poderoso del uso de contextos es la conexión a un *Docker Daemon* que se ejecuta en una máquina remota a través de SSH.

Requisitos Previos

Para crear un contexto SSH, se necesitan dos condiciones de seguridad:

1. **Acceso SSH:** Debe tenerse acceso a la máquina remota que tiene Docker instalado.
2. **Autenticación por Clave SSH:** El acceso debe ser configurado para usar **claves SSH públicas/privadas** y no contraseñas.

Paso 1: Verificación de Entornos y Contextos Locales

1. **Listar Contextos Locales:** El comando `docker context ls` mostrará los contextos disponibles, como el `default` y el de `desktop` (si se ha instalado). El asterisco (*) indica el contexto activo.

```
docker context ls
```

```
# Muestra el contexto 'default' con el asterisco (*)
```

2. **Verificar Entorno Local:** Ejecutar comandos de Docker (ej. `docker image ls`) en el contexto `default` listará los recursos del *daemon* nativo local.

Paso 2: Creación del Contexto Remoto

Se utiliza `docker context create` para definir la conexión al servidor remoto.

Se especifica la forma de conexión y la dirección.

- **Sintaxis Detallada:**

```
docker context create [NOMBRE_CONTEXTO] --docker  
"host=ssh://[USUARIO]@[IP_REMOTE]"
```

- **Ejemplo de Creación:**

```
docker context create remote-docker --docker  
host=ssh://demo@192.168.20.2
```

- *Se crea un nuevo contexto llamado `remote-docker` que se conectará al daemon de la máquina remota a través de SSH.*

Paso 3: Activación y Uso del Contexto

Se utiliza el comando `docker context use` para cambiar el punto de conexión de la CLI.

1. **Cambiar el Contexto Activo:**

```
docker context use remote-docker
```

El asterisco ahora se moverá al contexto `remote-docker` en la lista.

2. **Comprobar la Conexión Remota:** Cualquier comando Docker ejecutado ahora se dirigirá al servidor remoto.

```
docker image ls  
# Muestra ahora las imágenes del servidor remoto (ej.  
'kindest').
```

3. **Ejecución de Tareas Remotas:** Todas las operaciones, como la construcción de imágenes, se llevan a cabo en el host remoto.

```
# Enviar contexto local y construir la imagen en el host  
remoto (192.168.20.2)  
docker build -t demo .
```

4. *Si la máquina remota no tiene las imágenes base en caché, las descargará antes de la construcción.*
5. **Verificación de la Persistencia:** Al conectarse directamente al servidor remoto a través de SSH, se puede confirmar que la imagen **demo** ha sido creada en ese sistema.

Paso 4: Retorno al Contexto Local

Para volver a trabajar con la instalación nativa, simplemente se cambia el contexto al valor por defecto.

```
docker context use default
```

```
docker image ls # Muestra de nuevo las imágenes locales
```

III. Conclusiones y Relevancia en Ciberseguridad

El uso de Contextos de Docker es una práctica de gestión muy conveniente y eficiente:

- **Flexibilidad Operacional:** Permite a los desarrolladores y equipos de operaciones trabajar con entornos distribuidos sin necesidad de cambiar constantemente parámetros de conexión o usar scripts complejos.
- **Alternativa a la Distribución:** Los contextos facilitan la creación y el movimiento de imágenes entre servidores **sin tener que depender de un Registro (Registry)** para cada paso. Se puede construir una imagen en un servidor de desarrollo y ejecutarla inmediatamente en ese mismo servidor o

en otro servidor remoto (si el contexto es correcto) sin el paso intermedio de **push** y **pull**. Sin embargo, para la **distribución a gran escala**, el uso de un Registro sigue siendo el método ideal y recomendado.

- **Entornos Consistentes:** El Contexto garantiza que el usuario siempre sepa con qué *daemon* y recursos está interactuando, lo que reduce errores y confusiones de estado.

Seguridad y Aislamiento de Red en Docker

En el ámbito de la ciberseguridad, la configuración de la red de contenedores es fundamental para lograr el **aislamiento** y la **microsegmentación**, minimizando así la superficie de ataque entre servicios. Docker ofrece varios *drivers* de red que determinan cómo se comunican los contenedores entre sí y con el mundo exterior.

I. Tipos de Redes por Defecto y Drivers

Docker instala varios *drivers* de red por defecto en el *host*, y es posible extender sus capacidades mediante *plugins* de terceros.

A. Redes Predeterminadas (Built-in Networks)

1. Red Bridge (**bridge**):

- **Descripción:** Es la red por **defecto** de Docker. Se crea automáticamente al iniciar Docker.
- **Aislamiento:** Los contenedores en esta red pueden **comunicarse entre sí** directamente (utilizando IP interna), pero están **aislados de las redes externas** del *host*, a menos que se configure un mapeo de puertos explícito (**-p**).
- **Uso:** Ideal para contenedores que necesitan comunicarse entre sí pero no requieren acceso directo a la red del *host*.

2. Red Host (**host**):

- **Descripción:** Los contenedores que se conectan a esta red **comparten el espacio de nombres de red con el host**.
- **Aislamiento:** Ofrece **cero aislamiento** a nivel de red, otorgando al contenedor acceso directo a las interfaces y servicios de red del *host*.
- **Rendimiento y Seguridad:** Ofrece potencialmente un mejor rendimiento (eliminando la capa NAT/Bridge), pero **sacrifica completamente la seguridad**. Se considera una **mala práctica** en la mayoría de los escenarios de producción.

3. Red None (**none**):

- **Descripción:** Los contenedores que se conectan a esta red se crean **sin ninguna interfaz de red** (solo la interfaz de *loopback*).
- **Aislamiento:** Total aislamiento. El contenedor **no tiene comunicación externa** (ni con otros contenedores, ni con el *host*, ni con Internet).
- **Uso:** Ideal para tareas que no requieren red y donde el aislamiento total es necesario (ej. procesamiento de datos sensibles o *sandboxing*).

B. Drivers Avanzados y de Terceros

1. **Redes Macvlan:** Permite que los contenedores tengan su **propia dirección MAC única**, haciendo que parezcan dispositivos físicos distintos en la red.
2. **Plugins de Terceros (*Third-Party Plugins*):** Docker admite *plugins* que extienden las capacidades de red, ofreciendo funciones avanzadas como **segmentación de red**, **cifrado de tráfico** y políticas de red más complejas (utilizados a menudo en orquestadores como Kubernetes).

II. Gestión de Redes y Aislamiento con **docker network**

La gestión de redes personalizadas es la clave para implementar la microsegmentación y el aislamiento de seguridad entre grupos de contenedores. Todas las operaciones de red se realizan con el comando de gestión **docker network**.

Comandos de Gestión de Redes

Comando	Propósito
docker network ls	Lista las redes disponibles en el contexto actual (bridge , host , none , y redes personalizadas).

<code>docker network create</code>	Crea una nueva red (por defecto, de tipo <i>bridge</i>).
<code>docker network rm</code>	Elimina una o más redes.
<code>docker network prune</code>	Elimina todas las redes no utilizadas.
<code>docker network connect</code>	Conecta un contenedor en ejecución a una red existente.
<code>docker network disconnect</code>	Desconecta un contenedor de una red.
<code>docker network inspect</code>	Muestra información detallada (incluyendo subredes e IPs asignadas) de una o más redes.

A. Demostración del Aislamiento Total con **none**

El modo de red **none** elimina la interfaz de red del contenedor.

1. Ejecutar contenedor en red **none**:

```
docker run -it --network none ubuntu
```

2. Verificar Interfaces de Red (Dentro del Contenedor):

```
ls -l /sys/class/net
```

Resultado: Solo se mostrará la interfaz de *loopback* (`lo`). La interfaz Ethernet (`eth0`) estará ausente, confirmando el aislamiento total.

B. Microsegmentación con Redes Bridge Personalizadas

La forma más efectiva de aislar contenedores dentro de un *host* es creando redes *bridge* personalizadas para microsegmentar las aplicaciones.

1. **Creación de Redes Aisladas:** Se crean dos redes *bridge* personalizadas, que por defecto no tienen comunicación directa entre sí.

```
docker network create red-a
```

```
docker network create red-b
```

2. **Despliegue de Contenedores Aislados:** Se inician dos contenedores en redes separadas. El *hostname* se establece para facilitar las pruebas de comunicación.

```
# Contenedor 1, en la Red A
```

```
docker run -it --hostname a1 --network red-a alpine
```

```
# Contenedor 2, en la Red B
```

```
docker run -it --hostname b1 --network red-b alpine
```

3. **Verificación del Aislamiento (No Comunicación):** Se comprueban las IPs internas asignadas a cada contenedor (ej. `ifconfig` dentro de Alpine). Se observa que las IPs pertenecen a segmentos de red distintos (ej. `red-a` en `172.18.x.x` y `red-b` en `172.19.x.x`).

Intento de comunicación de A1 a B1:

```
# Dentro de a1:
```

```
ping 172.19.0.2
```

```
# Resultado: La comunicación falla.
```

Conclusión: El contenedor **a1** **no puede comunicarse** con el contenedor **b1** porque están en redes Docker aisladas sin enrutamiento entre ellas.

4. **Verificación de la Comunicación (Dentro de la Misma Red):** Se inicia un tercer contenedor (**a2**) en la misma red que **a1**.

```
# Contenedor 3, en la Red A
```

```
docker run -it --hostname a2 --network red-a alpine
```

Intento de comunicación de A1 a A2:

```
# Dentro de a1:
```

```
ping 172.18.0.3 # La IP de a2
```

```
# Resultado: La comunicación es exitosa.
```

Conclusión: **Todos los contenedores dentro de la misma red personalizada pueden comunicarse entre sí**, pero quedan aislados de los contenedores en otras redes personalizadas.

III. Conclusiones para Ciberseguridad

- **Aislamiento por Defecto:** Los contenedores se ejecutan en redes internas aisladas, lo cual es la base de la seguridad en Docker. Para el acceso externo es indispensable el mapeo de puertos (**-p**).
- **Microsegmentación:** Crear redes *bridge* personalizadas con **docker network create** permite la **microsegmentación** de las aplicaciones. Esto es una buena práctica de seguridad que aísla componentes críticos (ej. base de datos) del *frontend* o de otros servicios menos sensibles, limitando el movimiento lateral de un atacante.

- **Red `host` (Riesgo):** La conexión directa a la red del *host* (`--network host`) rompe el aislamiento, compromete la seguridad y debe ser evitada.

Seguridad y Flexibilidad: Gestión de Variables de Entorno en Docker

Las **Variables de Entorno (ENV)** en Docker son un mecanismo fundamental para desacoplar la configuración de la aplicación de la imagen del contenedor. Permiten inyectar valores dinámicos (pares clave-valor) en el entorno del contenedor en **tiempo de ejecución**, afectando el comportamiento de los procesos que se ejecutan dentro. En el contexto de ciberseguridad, la gestión segura de estas variables es crítica, especialmente para datos sensibles.

I. Propósito y Beneficios de las Variables de Entorno

1. Desacoplamiento de la Configuración

- **Principio *Build Once, Deploy Anywhere***: La principal ventaja es la capacidad de construir la imagen de Docker **una sola vez** e implementarla en múltiples entornos (Desarrollo, Pruebas, Producción) sin necesidad de reconstruir la imagen para cada uno. Cada entorno utiliza un conjunto diferente de variables de entorno para personalizar el comportamiento del contenedor (ej. cambiando la URL de la base de datos o el nivel de *logging*).

2. Acceso y Visibilidad

- **Accesibilidad**: Las variables de entorno son accesibles por **cualquier proceso** que se ejecute dentro del contenedor.
- **Persistencia**: A diferencia de las variables de construcción (**ARG**), las variables de entorno definidas con **ENV** se **persisten** en la imagen final y se heredan en el contenedor en tiempo de ejecución.

II. Métodos para Proporcionar Variables de Entorno

Existen tres métodos principales para definir variables de entorno en un contenedor, cada uno con implicaciones de seguridad distintas.

A. Definición en el Dockerfile (**ENV**)

La instrucción **ENV** permite establecer variables de entorno que se almacenarán dentro de la capa de la imagen.

Sintaxis:

```
ENV <clave>=<valor>
```

```
# 0 bien
```

```
ENV <clave> <valor>
```

*Es posible definir múltiples variables en una sola instrucción **ENV** usando \ para la legibilidad.*

Ejemplo Práctico (NO SEGURO): Usar **ENV** para pasar la contraseña de **root** de MySQL.

```
FROM mysql
```

```
ENV MYSQL_ROOT_PASSWORD=mysecretpassword
```

Advertencia de Seguridad Crucial: NUNCA se deben incluir datos sensibles (contraseñas, *tokens*, claves API) directamente en el Dockerfile usando **ENV**.

- **Razón:** Cualquier persona que tenga acceso a la imagen (a través de un registro público o mediante **docker pull**) puede inspeccionar sus capas (usando **docker inspect** o **docker history**) y extraer el valor sensible en texto plano. Las variables **ENV** deben reservarse para datos no sensibles (descripciones, URLs públicas, etc.).

B. Definición en Línea de Comandos (**-e** o **--env**)

La forma más común y flexible de inyectar variables en tiempo de ejecución es a través del *flag* `-e` o `--env` del comando `docker run`.

Sintaxis:

```
docker run -e <VARIABLE>=<VALOR> <IMAGEN>
```

Para múltiples variables:

```
docker run -e VAR1=VAL1 -e VAR2=VAL2 <IMAGEN>
```

Ejemplo Práctico (INSEGURO, pero Común): Establecer una variable requerida por la imagen de MySQL:

```
docker run -it -e MYSQL_ALLOW_EMPTY_PASSWORD=true mysql
```

Advertencia de Seguridad: Aunque es más seguro que usar `ENV` en el Dockerfile, definir contraseñas directamente en la línea de comandos (ej. `-e MYSQL_ROOT_PASSWORD=secreto`) **es una mala práctica** en producción, ya que el comando completo queda registrado en el historial de *shell* del *host* y puede ser inspeccionado por otros usuarios o procesos. Este método debe limitarse a datos no sensibles o entornos de desarrollo *ad-hoc*.

C. Definición a Través de un Archivo (`--env-file`)

El método más recomendable para manejar múltiples variables de entorno no sensibles de forma centralizada es utilizar un archivo.

Sintaxis:

```
docker run -it --env-file <ruta/al/archivo> <IMAGEN>
```

Archivo de Entorno (`creds.txt`): El archivo contiene las variables en formato `clave=valor`, una por línea.

```
# Contenido de creds.txt
```

```
MYSQL_ALLOW_EMPTY_PASSWORD=true
```

```
DB_NAME=clientes_prod
```

```
LOG_LEVEL=INFO
```

Ejecución:

```
docker run -it --env-file creds.txt mysql
```

Seguridad: Este método es más limpio que pasar docenas de variables en la CLI, pero el archivo en sí (**creds.txt**) aún contiene los valores en texto plano y **debe ser excluido del repositorio de código** (usando **.gitignore**) y gestionado con cuidado.

III. Conclusión de Seguridad: Gestión de Datos Sensibles

En un contexto de ciberseguridad, el manejo de secretos es la conclusión más importante sobre las variables de entorno:

1. **Construir una vez, Desplegar Múltiples Veces:** La filosofía de usar variables de entorno para la configuración es una buena práctica y debe seguirse.
2. **Nunca Incluir Secretos en la Imagen:** Bajo ninguna circunstancia se deben incluir contraseñas o datos sensibles en el **Dockerfile (ENV)** o en el **código de la aplicación** dentro de la imagen. La imagen y sus ficheros son accesibles a cualquiera que pueda hacer **docker pull**.
3. **Almacenamiento Seguro:** Los datos sensibles siempre deben almacenarse en un **sitio seguro** y dedicado para la gestión de secretos (ej. **Docker Secrets**, HashiCorp Vault, AWS Secrets Manager, Azure Key Vault).

4. **Inyección en Tiempo de Ejecución:** Los secretos deben ser **inyectados** de forma segura en el contenedor en el momento exacto de su ejecución, a través de variables de entorno (utilizando sistemas de secretos) o volúmenes montados, nunca a través de la CLI o la imagen.

Interacción Avanzada y Análisis Forense Básico de Contenedores

Para un Máster en Ciberseguridad, la interacción con contenedores no se limita a su ejecución, sino a la capacidad de **inspeccionar, depurar y extraer datos** en tiempo real. Esta sesión se centra en las herramientas de la CLI de Docker para el control del ciclo de vida, la depuración activa (**exec**) y la extracción de artefactos (**cp**, **export**) de un contenedor en ejecución.

I. Control del Ciclo de Vida: Ejecución en Segundo Plano

Hasta ahora, muchos contenedores se han ejecutado en modo interactivo y adjunto (**-it**). Para procesos de servidor (bases de datos, servidores web), la práctica recomendada es ejecutarlos en segundo plano.

1. Ejecución Desligada (**-d**)

- **Opción **-d** (*Detached*):** Se utiliza para iniciar un contenedor en segundo plano (en modo *daemon* o desligado). Esto libera la terminal inmediatamente mientras el proceso del contenedor sigue activo.

```
docker run -d -e MYSQL_ROOT_PASSWORD=mysecret mysql
```

- **Verificación:** El comando **docker ps** se utiliza para listar **solo los contenedores que están en estado *running***.

```
docker ps
```

2. Visualización de Logs (**docker logs**)

El comando `docker logs` permite ver la salida estándar (`stdout`) y el error estándar (`stderr`) del proceso principal (PID 1) del contenedor.

- **Sintaxis:**

```
docker logs [NOMBRE_CONTENEDOR]
```

- *En Docker, solo el proceso con **PID 1** dentro del contenedor interactúa directamente con el Docker Daemon para el manejo de logs y el control del ciclo de vida.*

II. Interacción en Tiempo Real (`docker exec`)

El comando `docker exec` es crucial para la depuración y el análisis de seguridad, ya que permite ejecutar comandos **adicionales** dentro de un contenedor que ya está en funcionamiento, sin afectar su proceso principal (PID 1).

1. Ejecución de Comandos Únicos

Se pueden ejecutar comandos puntuales para inspección:

Sintaxis:

```
docker exec -it [NOMBRE_CONTENEDOR] <COMANDO> [ARGUMENTOS]
```

-it (Interactiva con TTY): Es necesario para comandos que requieren interacción o una sesión pseudo-terminal, como `bash` o `ls`.

Ejemplo de Inspección:

```
docker exec -it [NOMBRE_CONTENEDOR] ls -l /etc
```

Este comando muestra el contenido del directorio `/etc` del contenedor.

2. Acceso a una Shell Activa (Depuración)

La forma más común de usar `docker exec` es para abrir una *shell* de interacción (`bash` o `sh`) dentro del contenedor para una depuración activa.

Sintaxis para Shell:

```
docker exec -it [NOMBRE_CONTENEDOR] bash
```

Ejemplo Práctico (Debugging de Contraseña MySQL):

1. Abrir la *shell* dentro del contenedor MySQL.
2. Usar el cliente MySQL, pasando la contraseña inyectada con la variable de entorno:

```
mysql -p
(Enter mysecret password)
# La conexión exitosa confirma que la variable de
entorno se inyectó correctamente.
```

3. Para salir de la *shell* secundaria, se utiliza el comando `exit`. El proceso principal del contenedor (MySQL) **sigue en ejecución**.

III. Extracción de Datos y Forense Básico

En ciberseguridad, a menudo es necesario extraer configuraciones, logs, o binarios sospechosos de un contenedor comprometido para su análisis. Docker ofrece dos comandos para mover datos entre el contenedor y el *host*.

1. Copia Selectiva de Archivos (`docker cp`)

El comando `docker cp` es la herramienta de elección para copiar archivos o directorios específicos entre el contenedor y el *host* en ambas direcciones.

- **Extracción del Contenedor al Host:**

- **Propósito:** Útil para extraer ficheros de configuración (`nginx.conf`) para usarlos como base o para extraer *logs* y artefactos maliciosos.

- **Sintaxis:**

```
docker cp [NOMBRE_CONTENEDOR]:[RUTA_CONTENEDOR]
[RUTA_HOST]
```

- **Ejemplo (Extracción de Configuración Nginx):**

```
docker cp [NOMBRE_CONTENEDOR]:/etc/nginx/nginx.conf
.
```

- **Copia del Host al Contenedor:**

- **Propósito:** Útil para inyectar *scripts* de depuración o archivos temporales de prueba en un contenedor en ejecución.

- **Sintaxis:**

```
docker cp [RUTA_HOST]
[NOMBRE_CONTENEDOR]:[RUTA_CONTENEDOR]
```

- **Ejemplo (Inyección de un archivo):**

```
docker cp creds.txt [NOMBRE_CONTENEDOR]:/
```

2. Exportación Completa del Sistema de Ficheros (`docker export`)

El comando `docker export` permite extraer la **totalidad del sistema de ficheros** del contenedor en ejecución a un único archivo `tar` en el *host*.

- **Propósito:** Es extremadamente útil para el **análisis forense** de contenedores. Permite capturar el estado completo del sistema de ficheros de un contenedor (incluyendo cualquier cambio, ficheros temporales o artefactos dejados por un atacante) y analizarlo fuera del entorno de Docker.

- **Sintaxis:**

```
docker export -o [NOMBRE_ARCHIVO].tar [NOMBRE_CONTENEDOR]
```


- **Ejemplo Práctico (Captura Forense):**

```
docker export -o nginx-forense.tar [NOMBRE_CONTENEDOR]
```

```
# Descomprimir y analizar en el host
```

```
tar xvf nginx-forense.tar
```

IV. Conclusiones sobre la Interacción

La interacción con contenedores en tiempo de ejecución es crucial para el diagnóstico y la seguridad:

- **Separación de Modos:** Utilizar el modo **detached (-d)** para procesos de servidor y el modo **interactive (-it)** solo para *debugging* con **docker exec**.
- **Depuración Segura:** **docker exec** permite investigar un contenedor sin interrumpir su proceso principal, lo cual es vital para el *troubleshooting* de producción.
- **Extracción de Evidencias:** Los comandos **docker cp** y **docker export** son herramientas esenciales para extraer **artefactos de seguridad** y configuraciones para su posterior análisis en el *host*.

Persistencia de Datos en Contenedores Docker:

Seguridad y Almacenamiento

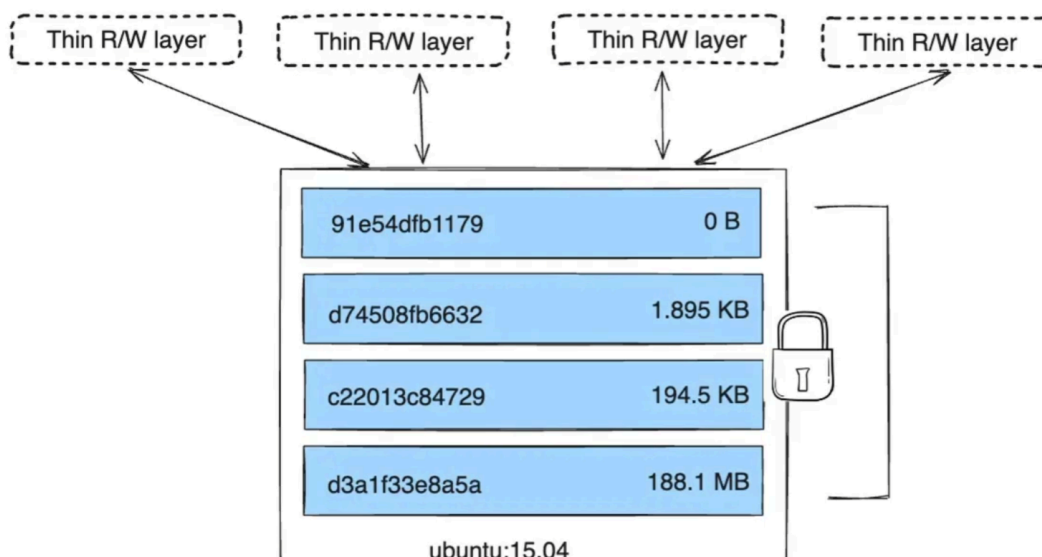
En el contexto de la ciberseguridad y la ingeniería de contenedores, la gestión de la persistencia de datos es crucial. La estructura inmutable de la imagen de Docker implica que los datos generados por un contenedor en ejecución son, por defecto, volátiles. Este capítulo detalla las tres opciones para la persistencia de datos, analizando sus implicaciones en la seguridad, la fiabilidad y el rendimiento.

I. Volatilidad de la Capa de Contenedor (Union File System)

La primera forma de "almacenamiento" es la propia capa de contenedor, la cual es volátil y no debe usarse para datos a largo plazo.

1. Arquitectura de Capas y Mutabilidad

- **Capas de la Imagen:** Las imágenes de Docker están compuestas por múltiples capas apiladas que son **inmutables** (solo lectura - R/O), representando una instantánea del sistema de archivos.
- **Capa del Contenedor:** Cuando se arranca un contenedor, Docker añade una **capa superior** que es **mutable** (Lectura/Escritura - R/W). Todos los cambios, archivos creados, *logs* y modificaciones ocurren en esta capa.



El esquema ilustra que las **Capas de la Imagen** (bloques azules inferiores) son inmutables (con un candado a la derecha), mientras que la **Capa de Contenedor** (Thin R/W layer) se añade en la parte superior y es donde se realizan los cambios.

2. Problema de la Persistencia

- **Volatilidad:** El principal problema es que una vez que el contenedor es borrado del sistema (`docker rm`), la capa mutable desaparece, y con ella, **todos los datos almacenados**.
- **Aislamiento:** Los datos almacenados en la capa R/W de un contenedor están aislados y **no se pueden compartir** con otros contenedores.

3. Inspección de Cambios (`docker diff`)

El comando `docker diff` permite inspeccionar los cambios (adiciones, modificaciones y eliminaciones) que se han realizado en la capa del contenedor desde su inicio:

- **Sintaxis:**

```
docker diff [NOMBRE_CONTENEDOR]
```

- **Códigos de Estado:**

- **A: Added** (Añadido)
- **C: Changed** (Cambiado)
- **D: Deleted** (Borrado)

- **Procedimiento Práctico:**

- Arrancar un contenedor en modo *detached*: `docker run -d --name layer nginx`.
- Hacer un cambio (ej. `touch newfile`, `rm 50x.html`) dentro del contenedor (`docker exec -it layer bash`).
- Inspeccionar los cambios: `docker diff layer`.

- El resultado mostrará A `/newfile` y D `/usr/share/nginx/html/50x.html`, además de que los directorios modificados aparecerán como C.
-

II. Opciones para la Persistencia Externa

Para la persistencia de datos más allá del ciclo de vida del contenedor, Docker ofrece dos mecanismos principales que almacenan los datos en el sistema de archivos del *host*: **Bind Mounts** y **Volúmenes**.

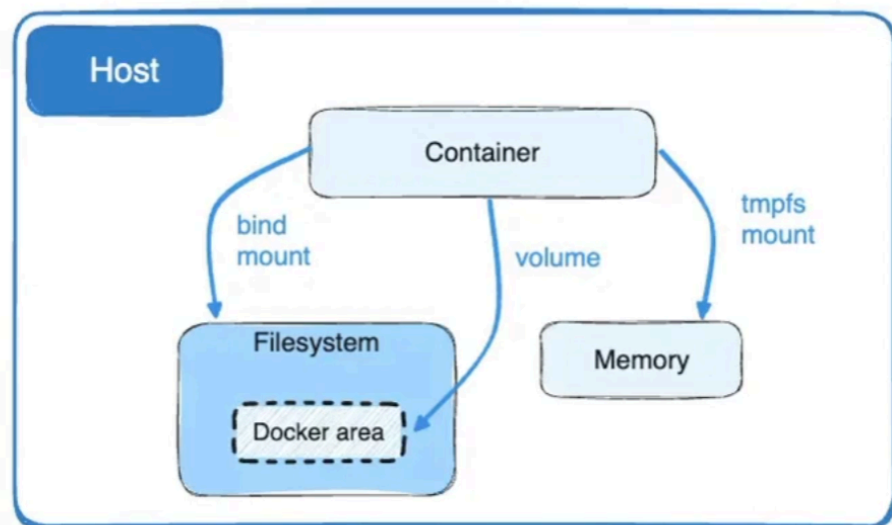
A. Montajes de Enlace (**Bind Mounts**)

El *bind mount* permite montar un archivo o directorio **existente** del sistema de archivos del *host* directamente dentro del contenedor.

Persistencia: Los datos persisten en el sistema de archivos del *host*. Si el contenedor se elimina o se recrea, el directorio en el *host* y sus datos permanecen intactos.

Uso Principal: Es la opción más común durante el **desarrollo** o la **programación**, ya que permite al desarrollador editar el código fuente en el *host* (con un editor local) y ver los cambios reflejados inmediatamente dentro del contenedor.

Riesgo de Seguridad: Aunque ofrece flexibilidad, el *bind mount* puede **comprometer la seguridad del host**. Si un contenedor es capaz de montar directorios sensibles del *host* (ej. el directorio raíz `/`), un atacante podría hacer un **chroot** o apoderarse del sistema completo. Por esta razón, su uso está restringido y es limitado en entornos como Docker Desktop.



El diagrama muestra la conexión **bind mount** que va directamente desde el **Contenedor** a una ruta arbitraria en el **Filesystem** del **Host**.

- **Sintaxis Práctica (-v):**

```
docker run -it -v [RUTA_HOST]:[RUTA_CONTENEDOR]
python:3.10 bash
```

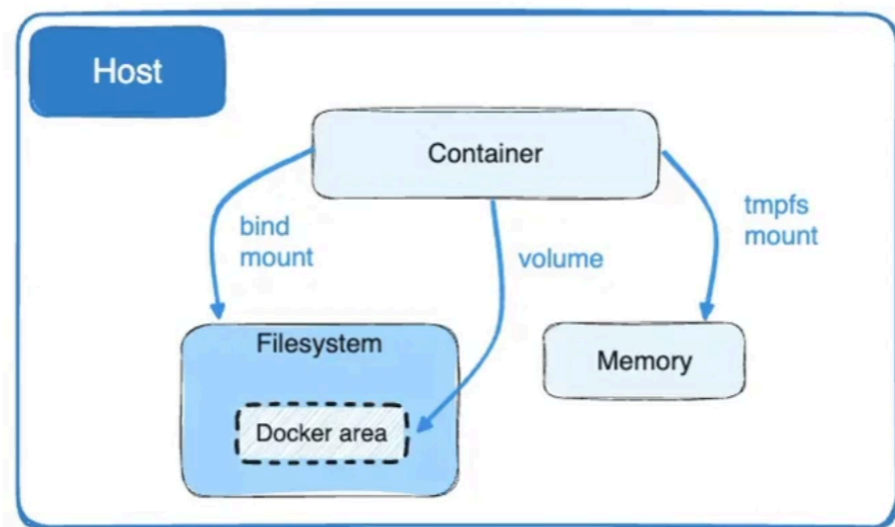
- **-v ./app:** Monta el directorio actual del *host* (.) en el directorio */app* del contenedor.

B. Volúmenes de Docker (Volumes)

Los volúmenes son la **forma preferida y más segura** de persistir datos para los contenedores, siendo gestionados directamente por Docker.

- **Gestión por Docker:** Los volúmenes se almacenan en un área específica del sistema de archivos del *host* (el "Docker area"), pero la ubicación física es gestionada por el *daemon* de Docker, no por el usuario.
- **Ventajas:** Proporcionan mejor rendimiento y fiabilidad que los *bind mounts*, y pueden ser **compartidos y reutilizados** entre múltiples contenedores.
- **Tipos:**

- **Volúmenes Nombrados (*Named Mounts*):** Se crean con un nombre explícito (ej. `vol1`), lo que facilita su referenciación y uso compartido.
- **Volúmenes Sin Nombre/Anónimos (*Unnamed/Anonymous Mounts*):** Docker les asigna un ID aleatorio y largo. Se crean implícitamente sin especificar un nombre.



El diagrama ilustra la conexión **volume** desde el **Contenedor** hacia el **Filesystem** del **Host**, específicamente dentro del área demarcada como **Docker area** (área gestionada).

- **Comandos de Gestión de Volúmenes:**

```
docker volume create vol1      # Crear un volumen
                                nombrado
docker volume ls                # Listar volúmenes
```

- **Montaje de Volúmenes Nombrados (Uso Compartido):**

```
# Contenedor 1 monta y crea archivo
docker run -it -v vol1:/data ubuntu
```

```
# Contenedor 2 monta el mismo volumen y ve el archivo
docker run -it -v vol1:/mydata alpine
```

- **Creación de Volúmenes Anónimos:**

- **Opción 1: En Línea de Comandos:** Especificando solo la ruta de montaje en el contenedor:

```
docker run -it -v /data2 ubuntu # Docker crea un
volumen anónimo con ID largo.
```

- **Opción 2: En Dockerfile:** Utilizando la instrucción **VOLUME**. Esto indica a Docker que se debe crear un volumen anónimo cuando se ejecute el contenedor a partir de esta imagen.

```
FROM ubuntu
```

```
VOLUME /data3
```

C. Montajes **Tmpfs**

Un *tmpfs mount* (ilustrado en el esquema) se utiliza para almacenar datos **solo en la memoria RAM** del *host*. Estos datos son completamente **volátiles** y se pierden inmediatamente cuando el contenedor se detiene o se elimina. Se usan para almacenar datos temporales muy sensibles o caché que nunca deben tocar el disco.